Py_DAW

Python Scripting Handbuch

Version 0.0.20.798 — ChronoScaleStudio

Vollständige API-Referenz für die Live-Scripting-Console

Die erste Open-Source-DAW mit nativer Python-Scripting-API

1. Einführung

Py_DAW ist die erste Digital Audio Workstation mit einer nativen Python-Scripting-Console. Du kannst Python-Code direkt in der DAW eingeben und damit Tracks, Clips, Mixer-Einstellungen und Transport in Echtzeit steuern. Jede Änderung wird sofort in der GUI sichtbar.

1.1 Console öffnen

Die Python Console erreichst du über:

```
Menü → Projekt → 🐍 Python Console…
Tastenkürzel: Ctrl+Alt+P
```

Die Console erscheint als Dock-Widget am unteren Bildschirmrand. Du kannst sie frei verschieben, andocken oder abreißen.

1.2 Erste Schritte

Tippe Python-Code in das Eingabefeld und drücke Enter:

```
>>> project.bpm = 140
>>> tracks[0].volume = 0.6
>>> print(project)
<Project 'Mein Song' bpm=140 tracks=8 clips=24>
```

💡 *Alles was du änderst wird sofort in der GUI angezeigt — Fader bewegen sich, BPM-Anzeige aktualisiert sich.*

1.3 Mehrzeiliger Code

Für längere Scripts benutze den Multi-Line-Editor unter der Eingabezeile:

```
for t in tracks:
    if t.kind != 'master':
        t.volume = 0.75
        t.pan = 0.0
print('Alle Tracks auf 0.75 gesetzt')
```

Klicke '▶️ Script ausführen' um den gesamten Block auszuführen.

2. API-Objekte — Übersicht

Die Scripting Console stellt folgende Objekte bereit, die direkt auf das geladene Projekt wirken:

Objekt	Beschreibung
project	Projekt-Einstellungen (BPM, Name, Sample-Rate, Taktart)
tracks	Liste aller Tracks — indexierbar per Nummer oder Name
clips	Liste aller Clips (Audio + MIDI)
mixer	Shortcut-Funktionen für Mute/Solo/Volume
transport	Wiedergabe-Steuerung (Play/Stop/Pause/Seek)
math	Python math-Modul (sin, cos, pi, etc.)

2.1 project — Projekt-Einstellungen

Das project-Objekt gibt dir Zugriff auf die globalen Projekt-Parameter:

Attribut	Typ	Beschreibung
project.bpm	float (r/w)	Tempo in BPM (20.0–999.0)
project.name	str (r/w)	Projektname
project.sample_rate	int (r)	Sample-Rate (44100/48000/96000)
project.time_signature	str (r/w)	Taktart ('4/4', '3/4', '6/8')
project.track_count	int (r)	Anzahl Tracks
project.clip_count	int (r)	Anzahl Clips

Methoden:

Methode	Beschreibung
project.save()	Projekt auf Disk speichern

Beispiele:

```
>>> project.bpm = 128
>>> project.name = 'Mein neuer Song'
>>> project.time_signature = '3/4'
>>> project.save()
'Project saved.'
>>> print(project)
<Project 'Mein neuer Song' bpm=128 tracks=8 clips=24>
```

2.2 tracks — Track-Liste

Das tracks-Objekt ist eine Liste aller Tracks im Projekt. Du kannst per Index, per Name oder per Iteration darauf zugreifen.

Zugriffsmethoden:

Syntax	Beschreibung
tracks[0]	Erster Track (Index)
tracks[-1]	Letzter Track
tracks['Bass']	Track mit Name 'Bass' (exakter Match)
tracks.by_name('bass')	Track suchen (case-insensitive)
tracks.by_kind('instrument')	Alle Instrument-Tracks
len(tracks)	Anzahl Tracks
for t in tracks:	Über alle Tracks iterieren

Track-Attribute:

Jeder Track hat folgende Attribute, die du lesen und schreiben kannst:

Attribut	Typ	Beschreibung
t.name	str	Track-Name
t.kind	str	Track-Typ: 'audio', 'instrument', 'bus', 'master', 'fx', 'group'
t.volume	float	Lautstärke 0.0–1.0 (0.8 = Standard)
t.pan	float	Panorama -1.0 (links) bis +1.0 (rechts)
t.muted	bool	True = stummgeschaltet
t.solo	bool	True = solo (nur dieser Track hörbar)
t.record_arm	bool	True = aufnahmebereit
t.monitor	bool	True = Input-Monitoring aktiv
t.instrument_enabled	bool	True = Instrument aktiv, False = bypassed
t.plugin_type	str/None	Instrument-Typ: 'sampler', 'drum_machine', 'sf2', None
t.automation_mode	str	'off', 'read', 'write'
t.channel_config	str	'stereo' oder 'mono'
t.sidechain_source_id	str	Track-ID der Sidechain-Quelle
t.id	str (r)	Eindeutige Track-ID (z.B. 'trk_abc123')

Beispiele:

```
# Track per Index
>>> tracks[0].volume = 0.5
>>> tracks[0].name = 'Kick Drum'

# Track per Name
```

```
>>> tracks['Bass'].pan = -0.3
>>> tracks['Bass'].muted = True

# Alle Tracks durchgehen
>>> for t in tracks:
...     print(f'{t.name}: vol={t.volume:.2f} pan={t.pan:+.2f}')
Kick Drum: vol=0.50 pan=+0.00
Bass: vol=0.80 pan=-0.30
...

# Nur Instrument-Tracks
>>> for t in tracks.by_kind('instrument'):
...     t.volume = 0.7

# Track-Liste anzeigen
>>> print(tracks)
TrackList (8 tracks):
  [0] Kick Drum (audio)
  [1] Bass (instrument)
  ...
```

2.3 clips — Clip-Liste

Das clips-Objekt enthält alle Audio- und MIDI-Clips im Projekt.

Syntax	Beschreibung
clips[0]	Erster Clip
clips.on_track('Bass')	Alle Clips auf dem Track 'Bass'
len(clips)	Anzahl Clips
for c in clips:	Über alle Clips iterieren

Clip-Attribute:

Attribut	Typ	Beschreibung
c.kind	str	'audio' oder 'midi'
c.label	str	Clip-Beschriftung
c.start_beats	float	Startposition in Beats
c.length_beats	float	Dauer in Beats
c.gain	float	Clip-Lautstärke (linear, 1.0 = unity)
c.pan	float	Clip-Panorama (-1.0 bis +1.0)
c.pitch	float	Pitch-Shift in Halbtönen
c.muted	bool	True = Clip stummgeschaltet
c.reversed	bool	True = rückwärts abspielen
c.stretch	float	Time-Stretch-Faktor (1.0 = original)
c.fade_in_beats	float	Fade-In-Länge in Beats
c.fade_out_beats	float	Fade-Out-Länge in Beats
c.track_id	str (r)	Auf welchem Track liegt der Clip
c.id	str (r)	Eindeutige Clip-ID

Beispiele:

```
# Clip verschieben
>>> clips[0].start_beats = 8.0

# Alle Clips auf einem Track leiser machen
>>> for c in clips.on_track('Vocals'):
...     c.gain = 0.7

# Clip mutet
>>> clips[3].muted = True

# Fade hinzufügen
>>> clips[0].fade_in_beats = 0.5
>>> clips[0].fade_out_beats = 1.0
```

2.4 transport — Wiedergabe-Steuerung

Methode / Attribut	Beschreibung
transport.play()	Wiedergabe starten
transport.stop()	Wiedergabe stoppen
transport.pause()	Wiedergabe pausieren
transport.position	Playhead-Position in Beats (r/w)
transport.is_playing	True wenn Wiedergabe läuft (r)

Beispiele:

```
>>> transport.play()
'▶️ Playing'
>>> transport.position = 32.0   # Springe zu Beat 32
>>> transport.stop()
'⏹️ Stopped'
>>> print(transport)
<Transport ⏹️ Stopped at beat 32.0>
```

2.5 mixer — Mixer-Shortcuts

Bequeme Funktionen für häufige Mixer-Operationen:

Methode	Beschreibung
mixer.solo('Drums')	Drums solo schalten (alle anderen unsolo)
mixer.unsolo_all()	Solo von allen Tracks entfernen
mixer.mute('Bass')	Bass stummschalten
mixer.unmute_all()	Alle Tracks entmuten
mixer.set_all_volume(0.75)	Alle Tracks auf gleiche Lautstärke

Beispiel:

```
>>> mixer.solo('Vocals')
'Solo: Vocals'
>>> mixer.unsolo_all()
'All tracks unsoloed'
>>> mixer.set_all_volume(0.75)
'All tracks volume → 0.75'
>>> print(mixer)
Mixer:
    Drums                vol=0.75 pan=+0.00
    Bass                  vol=0.75 pan=+0.00
    ...
```

3. Globale Funktionen

Neben den Objekten stehen dir globale Hilfsfunktionen zur Verfügung:

Funktion	Beschreibung
add_track(name, kind)	Neuen Track hinzufügen
remove_track(name)	Track per Name löschen
add_clip(track, start, length, kind, label)	Clip zu einem Track hinzufügen
sleep(seconds)	Warten (max 10 Sekunden)
help() / help_api()	API-Hilfe anzeigen
print(...)	Ausgabe in die Console

3.1 add_track(name, kind)

Erstellt einen neuen Track im Projekt.

Parameter	Typ	Beschreibung
name	str	Name des neuen Tracks
kind	str	'audio', 'instrument', 'bus', 'fx', 'group'

```
>>> add_track('Synth Lead', kind='instrument')
"Track 'Synth Lead' (instrument) added"

>>> add_track('Reverb Bus', kind='bus')
"Track 'Reverb Bus' (bus) added"
```

3.2 remove_track(name)

```
>>> remove_track('Synth Lead')
"Track 'Synth Lead' removed"
```

⚠️ *Achtung: Entfernt den Track und alle Clips darauf!*

3.3 add_clip(track, start, length, kind, label)

Parameter	Typ	Standard
track_name	str	(erforderlich)
start	float	0.0
length	float	4.0
kind	str	'midi'
label	str	(auto)

```
>>> add_clip('Bass', start=0.0, length=8.0, kind='midi', label='Intro')
"Clip added on 'Bass' at beat 0.0"
```

4. Praxis-Rezepte

Fertige Scripts für häufige Aufgaben — kopiere sie in den Multi-Line-Editor und führe sie aus.

4.1 Gain Staging (alle Tracks auf -6 dB)

```
import math
target_db = -6.0
target_linear = 10 ** (target_db / 20.0)
count = 0
for t in tracks:
    if t.kind != 'master':
        t.volume = target_linear
        count += 1
print(f'{count} Tracks auf {target_db} dB gesetzt ({target_linear:.3f} linear)')
```

4.2 Leere Tracks stummschalten

```
muted = 0
for t in tracks:
    track_clips = clips.on_track(t.name)
    if not track_clips and t.kind != 'master':
        t.muted = True
        muted += 1
print(f'{muted} leere Tracks stummgeschaltet')
```

4.3 8-Track-Template erstellen

```
template = [
    ('Drums', 'instrument'),
    ('Bass', 'instrument'),
    ('Keys', 'instrument'),
    ('Guitar', 'audio'),
    ('Vocals', 'audio'),
    ('Pad', 'instrument'),
    ('FX', 'bus'),
    ('Lead', 'instrument'),
]
for name, kind in template:
    add_track(name, kind=kind)
print(f'{len(template)} Tracks erstellt')
```

4.4 Alle Clips um 4 Beats verschieben

```
shift = 4.0  # Beats
count = 0
for c in clips:
    c.start_beats = c.start_beats + shift
    count += 1
```

```
print(f'{count} Clips um {shift} Beats verschoben')
```

4.5 Mix-Snapshot drucken

```
print(f'Projekt: {project.name}  BPM: {project.bpm}  SR: {project.sample_rate}')
print(f'Tracks: {project.track_count}  Clips: {project.clip_count}')
print()
print(mixer)
```

4.6 Stereo-Pan verteilen (Auto-Pan)

```
import math
instrument_tracks = tracks.by_kind('instrument')
n = len(instrument_tracks)
for i, t in enumerate(instrument_tracks):
    # Gleichmäßig von links nach rechts verteilen
    t.pan = -1.0 + (2.0 * i / max(1, n - 1))
    print(f'{t.name}: pan={t.pan:+.2f}')
```

4.7 BPM verdoppeln / halbieren

```
# Verdoppeln
old = project.bpm
project.bpm = old * 2
print(f'BPM: {old} → {project.bpm}')

# Halbieren
project.bpm = project.bpm / 2
```

5. Sicherheit & Einschränkungen

Die Scripting Console läuft in einer Sandbox mit eingeschränkten Python-Builtins. Das schützt dein System vor versehentlichen oder bösartigen Operationen.

5.1 Erlaubt

Alle Standard-Datentypen und -Funktionen: int, float, str, list, dict, set, tuple, range, len, max, min, sorted, enumerate, zip, map, filter, print, round, abs, sum, any, all, math-Modul.

5.2 Blockiert

Folgende Operationen sind aus Sicherheitsgründen nicht verfügbar:

Blockiert	Grund
import os / sys / subprocess	Kein Zugriff auf Betriebssystem
open() / Datei-I/O	Keine direkte Dateimanipulation
exec() / eval() / compile()	Kein dynamischer Code
__import__()	Keine Import-Umgehung
Netzwerk (socket, urllib)	Kein Netzwerkzugriff

⚠️ *Die Console ist für DAW-Steuerung gedacht, nicht für allgemeine Programmierung. Für erweiterte Scripts nutze einen externen Python-Editor.*

5.3 sleep()-Limit

Die sleep()-Funktion ist auf maximal 10 Sekunden begrenzt, um die DAW nicht zu blockieren.

6. Script-Bibliothek

Die Console enthält eine eingebaute Script-Bibliothek (Tab '📚 Scripts'). Dort findest du vorgefertigte Scripts, die du per Doppelklick in den Editor laden und ausführen kannst.

Eingebaute Scripts:

Script	Beschreibung
Gain Staging	Alle Non-Master-Tracks auf Volume 0.75
Mute Empty Tracks	Tracks ohne Clips stummschalten
Project Info	Projekt-Informationen + Mixer-Status anzeigen
Double BPM	Projekt-Tempo verdoppeln
Create 8-Track Template	Standard 8-Track Setup erstellen

Du kannst eigene Scripts speichern und sie in der Bibliothek wiederfinden. Die Scripts werden pro Session gespeichert.

7. Tastaturkürzel

Taste	Aktion
Ctrl+Alt+P	Console ein/ausblenden
Enter	Einzeiligen Code ausführen
Pfeiltaste ↑	Vorheriger Befehl (History)
Pfeiltaste ↓	Nächster Befehl (History)

8. Quick Reference Card

Die wichtigsten Befehle auf einen Blick:

```
# Projekt
project.bpm = 120
project.name = 'Mein Song'
project.save()

# Tracks
tracks[0].volume = 0.8
tracks['Bass'].muted = True
tracks.by_name('drums').solo = True
add_track('Neuer Track', kind='instrument')
remove_track('Alter Track')

# Clips
clips[0].start_beats = 4.0
```

```
clips[0].gain = 0.7
clips.on_track('Vocals')[0].muted = True
add_clip('Bass', start=0, length=8)

# Mixer
mixer.solo('Drums')
mixer.unsolo_all()
mixer.mute('FX')
mixer.unmute_all()
mixer.set_all_volume(0.75)

# Transport
transport.play()
transport.stop()
transport.position = 32.0

# Info
print(project)
print(tracks)
print(mixer)
help()
```